

322. Coin Change Medium

Acceptance: 47.8% | [LeetCode](#)

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
**Input:** coins = [1,2,5], amount = 11
**Output:** 3
**Explanation:** 11 = 5 + 5 + 1
```

Example 2:

```
**Input:** coins = [2], amount = 3
**Output:** -1
```

Example 3:

```
**Input:** coins = [1], amount = 0
**Output:** 0
```

Constraints:

- `1 <= coins.length <= 12`
- `1 <= coins[i] <= 231 - 1`
- `0 <= amount <= 104`

Claude Opus Solution

Coin Change — Complete Solution Guide

1. Intuition

The key insight is that this is an **unbounded knapsack** problem (we can use each coin infinitely).

The crucial observation: **the minimum coins to make amount `i` depends on smaller subproblems**. If we want to form amount `i`, we can pick any coin `c`, then we only need to solve for the remaining `i - c`. This gives the recurrence:

```
minCoins(i) = 1 + min(minCoins(i - c)) for each coin c where c <= i
```

We add `1` because picking coin `c` is one coin used, plus whatever was optimal for the rest.

This **optimal substructure** (optimal solution built from optimal sub-solutions) plus **overlapping subproblems** (e.g., `minCoins(5)` is needed many times) is the signature of **Dynamic Programming**.

A greedy approach (always pick the largest coin) **fails**, e.g., `coins=[1,3,4]`, `amount=6`: greedy gives `4+1+1=3` coins, but optimal is `3+3=2` coins.

2. Approach (Pseudocode)

- Create dp array of size `(amount + 1)`, filled with infinity
 - `dp[i]` = fewest coins needed to make amount `i`
- Base case: `dp[0] = 0` (zero coins needed to make amount 0)
- For each amount `i` from 1 to amount:
 - For each coin in `coins`:
 - if `coin <= i` and `dp[i - coin]` is reachable:
 - `dp[i] = min(dp[i], dp[i - coin] + 1)`
- If `dp[amount]` is still infinity $\rightarrow$ return `-1` (impossible)
 - else return `dp[amount]`

3. Python Solution

```
from typing import List
```

```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        # dp[i] = minimum number of coins needed to make up amount i.
        # Initialize all values to "infinity" meaning "unreachable yet".
        # Size is amount + 1 so indices range from 0 to amount inclusive.
        dp = [float("inf")] * (amount + 1)
```

```

# Base case: it takes 0 coins to make amount 0.
dp[0] = 0

# Build up solutions for every amount from 1 up to the target.
for i in range(1, amount + 1):
    # Try using each coin denomination as the "last coin" used.
    for coin in coins:
        # The coin can only be used if it doesn't exceed amount i.
        if coin <= i:
            # If we use this coin, the remaining amount is (i - coin).
            # dp[i - coin] coins solve the rest; +1 accounts for this coin.
            # Keep the minimum across all coin choices.
            dp[i] = min(dp[i], dp[i - coin] + 1)

# If dp[amount] was never updated, the amount is unreachable.
return dp[amount] if dp[amount] != float("inf") else -1

```

Walkthrough: coins=[1,2,5], amount=11

```

dp[0] = 0
dp[1] = dp[0]+1 = 1          (1)
dp[2] = dp[0]+1 = 1          (2)
dp[3] = dp[1]+1 = 2          (1+2)
dp[4] = dp[2]+1 = 2          (2+2)
dp[5] = dp[0]+1 = 1          (5)
dp[6] = dp[5]+1 = 2          (5+1)
...
dp[11] = dp[6]+1 = 3          (5+5+1) ✓

```

4. Complexity Analysis

Let n = amount and m = number of coins.

- **Time Complexity:** $O(n \times m)$

We fill n entries of the dp array, and for each entry we iterate over all m coins. Each inner operation is $O(1)$.

- **Space Complexity:** $O(n)$

We store one dp array of size `amount + 1`. This is independent of the number of coins.

5. Alternative Approaches

A) Top-Down Memoization (Recursion + Cache)

Same recurrence but solved recursively, caching results to avoid recomputation. Often more intuitive but adds recursion overhead.

```

from functools import lru_cache

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        @lru_cache(maxsize=None)
        def dp(rem):
            if rem == 0:
                return 0
            if rem < 0:
                return float("inf")
            # Try every coin and take the best (fewest-coin) result.
            return min((dp(rem - c) + 1 for c in coins), default=float("inf"))

        result = dp(amount)
        return result if result != float("inf") else -1

```

- Same $O(n \times m)$ time, $O(n)$ space (plus recursion stack).

B) BFS (Shortest Path Interpretation)

Treat amounts as graph nodes; each coin is an edge of weight 1. The shortest path from 0 to `amount` = fewest coins. BFS naturally finds the minimum-edge path. Can be faster in practice due to early termination.

```

from collections import deque

```

```

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        if amount == 0:
            return 0
        visited = {0}
        queue = deque([(0, 0)]) # (current_amount, coins_used)
        while queue:
            curr, used = queue.popleft()
            for c in coins:
                nxt = curr + c
                if nxt == amount:
                    return used + 1 # found target with min coins
                if nxt < amount and nxt not in visited:
                    visited.add(nxt)
                    queue.append((nxt, used + 1))
        return -1

```

- Worst-case $O(n \times m)$ time, $O(n)$ space.

C) Greedy — ❌ Does NOT work

Sorting coins descending and always taking the largest fails on cases like `coins=[1,3,4]`, `amount=6`. **Avoid greedy** unless coins form a "canonical" system (like standard US currency).

Recommended: The **bottom-up DP** (main solution) is the cleanest, avoids recursion limits, and is the standard expected answer in interviews.

Codex Solution

1. Intuition

This is a classic **Dynamic Programming / Unbounded Knapsack** problem.

We want the minimum number of coins needed to make every amount from `0` to `amount`.

Key idea:

If we already know the minimum coins needed to make amount:

```
i - coin
```

then we can make amount `i` by adding one `coin`.

So:

```
dp[i] = min(dp[i], dp[i - coin] + 1)
```

where:

```
dp[i] = minimum number of coins needed to make amount i
```

Base case:

```
dp[0] = 0
```

because zero coins are needed to make amount `0`.

If after processing all coins, `dp[amount]` is still infinity, then it is impossible to make that amount.

2. Approach

Pseudocode

```

function coinChange(coins, amount):
    create dp array of size amount + 1
    fill dp with infinity

    dp[0] = 0

    for current_amount from 1 to amount:
        for coin in coins:
            if coin <= current_amount:

```

```

        dp[current_amount] = min(
            dp[current_amount],
            dp[current_amount - coin] + 1
        )

    if dp[amount] is infinity:
        return -1

    return dp[amount]

```

3. Python Solution

```

from typing import List

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        # dp[i] represents the minimum number of coins needed
        # to make amount i.
        #
        # Initialize every value as infinity because initially
        # we assume no amount can be formed.
        dp = [float("inf")] * (amount + 1)

        # Base case:
        # It takes 0 coins to make amount 0.
        dp[0] = 0

        # Build the answer for every amount from 1 to amount.
        for current_amount in range(1, amount + 1):

            # Try using each coin as the last coin chosen.
            for coin in coins:

                # We can only use this coin if it does not exceed
                # the current amount we are trying to form.
                if coin <= current_amount:

                    # If we use this coin, then we need to have already
                    # formed current_amount - coin.
                    #
                    # The total coins would be:
                    # dp[current_amount - coin] + 1
                    #
                    # Take the minimum over all possible coins.
                    dp[current_amount] = min(
                        dp[current_amount],
                        dp[current_amount - coin] + 1
                    )

        # If dp[amount] is still infinity, then the amount cannot be formed.
        if dp[amount] == float("inf"):
            return -1

        return dp[amount]

```

4. Complexity Analysis

Let:

```

n = amount
m = number of coin denominations

```

Time Complexity

$O(n * m)$

We compute the answer for every amount from 1 to amount.

For each amount, we try every coin.

Therefore:

```
amount * len(coins)
```

Space Complexity

```
O(n)
```

We use a DP array of size:

```
amount + 1
```

So the space complexity is:

```
O(amount)
```

5. Alternative Approaches

1. Top-Down DP with Memoization

Instead of building from `0` up to `amount`, we can recursively solve:

```
minCoins(remaining_amount)
```

For each recursive call, try subtracting every coin.

Memoization prevents recomputing the same remaining amount many times.

Time complexity is also:

```
O(amount * len(coins))
```

2. BFS Shortest Path

You can think of each amount as a node.

From amount `x`, you can go to:

```
x + coin
```

for every coin.

We want the shortest number of steps to reach `amount` from `0`.

This works because each coin usage has equal cost `1`.

3. Greedy — Not Reliable

A greedy strategy like "always pick the largest coin possible" does **not** always work.

Example:

```
coins = [1, 3, 4]
amount = 6
```

Greedy picks:

```
4 + 1 + 1 = 3 coins
```

But the optimal answer is:

```
3 + 3 = 2 coins
```

So dynamic programming is needed.